Indian Institute of Information Technology Vadodara - International Campus Diu

Name: Saumya Joshi
Roll no.: 202311039
Branch: CSE
Batch: 2023
Subject: CS 268

Aim:

To perform binary multiplication by

1. Partial Sum
2. Booths Multiplication

C++ Code:

Partial Sum:

```cpp
#include <iostream>

using namespace std;

string binaryAddition(string a, string b, int &additions) {
   int i = a.length() - 1, j = b.length() - 1, carry = 0;
   string result = "";

   while (i >= 0 || j >= 0 || carry) {
       int sum = carry;
       if (i >= 0) sum += a[i--] - '0';
       if (j >= 0) sum += b[j--] - '0';

       result = char((sum % 2) + '0') + result;
       carry = sum / 2;

       additions++;
   }

   return result;
}
```

```cpp
string shiftLeft(string s, int shifts) {
    for (int i = 0; i < shifts; i++) {
        s += '0';
    }
    return s;
}

string binaryMultiplication(string a, string b, int &additions, int
&shifts) {
    string result = "0";

    for (int i = b.length() - 1; i >= 0; i--) {
        if (b[i] == '1') {
            string partialProduct = shiftLeft(a, b.length() - 1 - i);
            result = binaryAddition(result, partialProduct, additions);
        }
        shifts++;
    }

    return result;
}

int main() {
    string a, b;
    int additions = 0, shifts = 0;

    cout << "Enter first binary number: ";
    cin >> a;
    cout << "Enter second binary number: ";
    cin >> b;

    string product = binaryMultiplication(a, b, additions, shifts);
    cout << "Product: " << product << endl;
    cout << "Number of additions: " << additions << endl;
    cout << "Number of shifts: " << shifts << endl;
```

```
    return 0;
}
```

Booth's MUltiplication:

```
#include <vector>

using namespace std;

void add(vector<int> &a, const vector<int> &m, int n, int &add_count) {
    int carry = 0;
    for (int i = n - 1; i >= 0; i--) {
        int sum = a[i] + m[i] + carry;
        a[i] = sum % 2;
        carry = sum / 2;
    }
    add_count++;
}

void twos_complement(vector<int> &m) {
    int n = m.size();
    vector<int> temp(n);

    for (int i = 0; i < n; i++) {
        temp[i] = m[i] == 0 ? 1 : 0;
    }

    int carry = 1;
    for (int i = n - 1; i >= 0; i--) {
        int sum = temp[i] + carry;
        m[i] = sum % 2;
        carry = sum / 2;
    }
}
```

```
void shift_right(vector<int> &a, vector<int> &q, int &q0, int
&shift_count) {
    int n = a.size();
    q0 = q[n - 1];

    for (int i = n - 1; i > 0; i--) {
        q[i] = q[i - 1];
    }
    q[0] = a[n - 1];

    for (int i = n - 1; i > 0; i--) {
        a[i] = a[i - 1];
    }
    shift_count++;
}

vector<int> booth_multiplication(vector<int> m, vector<int> q) {
    int n = m.size();
    vector<int> product(2 * n);
    vector<int> a(n, 0);
    vector<int> m_neg = m;

    twos_complement(m_neg);

    int q0 = 0;
    int add_count = 0;
    int shift_count = 0;

    for (int i = 0; i < n; i++)
    {
        if (q[n - 1] == 1 && q0 == 0)
        {
            add(a, m_neg, n, add_count);
        }
```

```
        else if (q[n - 1] == 0 && q0 == 1)
        {
            add(a, m, n, add_count);
        }

        shift_right(a, q, q0, shift_count);
    }

    for (int i = 0; i < n; i++) {
        product[i] = a[i];
        product[i + n] = q[i];
    }

    cout << "Number of additions: " << add_count << endl;
    cout << "Number of shifts: " << shift_count << endl;

    return product;
}

void print_binary(const vector<int> &arr) {
    for (int bit : arr) {
        cout << bit;
    }
    cout << endl;
}

int main() {
    int n = 4;
    vector<int> m = {0, 1, 1, 1};
    vector<int> q = {0, 1, 1, 1};

    vector<int> result = booth_multiplication(m, q);

    cout << "Product: ";
    print_binary(result);
```

```
    return 0;
}
```

Output:

```
Enter first binary number:111

Enter second binary number:111

Product: 110001
Number of additions: 14
Number of shifts: 3
```

```
Number of additions: 2
Number of shifts: 4
Product: 00110001
```

Study on Booth's Multiplication Efficiency:

1. Reduction in Computation Time

○ Booth's algorithm reduces unnecessary additions by skipping consecutive 1's or 0's.

○ Fewer arithmetic operations mean better efficiency.

2. Hardware Optimization

○ Reduces the number of adders and shifts required.

○ Utilized in processors like ARM and x86 architectures.

The comparison will focus on:

- Number of adders used
- Number of logical shifts performed

Study on Partial Sum Multiplication Efficiency

1. Time Complexity:

○ Requires O(n) additions and shifts for an n-bit multiplier.

○ More 1s in the multiplier lead to more additions, making it less efficient.

2. Comparison with Booth's Algorithm:

○ Partial Sum Multiplication: Up to n additions.

○ Booth's Algorithm: Reduces additions (best case n/2), making it faster for numbers with long sequences of 1s.

3. Optimizations:

○ Carry-Save Adders (CSA): Reduces addition delay.

○ Wallace Tree Multiplication: Lowers additions from O(n) to O(log n).

○ Bitwise Operations: Improve efficiency in hardware.

4. Practical Use Cases:

○ Small-bit multiplication (8-bit, 16-bit): Partial sum method is acceptable.

○ Large-bit multiplication (32-bit, 64-bit): Booth’s algorithm or optimized hardware multipliers are preferred.

Performance Comparison:

Approach	Adders Used	Logical Shifts
Partial Sum Multiplication	O(n)	O(n)
Booth's Algorithm	O(log n)	O(log n)